Superglobal, Form, GET, POST, Cookie, Session, Include, Require

- Md. Morshedul Arefin

PHP Form (using post method)

HTML:

```
<form action="process.php" method="post">
Name: <input type="text" name="name"><br>
Email: <input type="email" name="email"><br>
<button type="submit">Submit</button>
</form>
```

URL After Submitting:

http://localhost/process.php

process.php

```
<?php
$name = $_POST['name'];
$email = $_POST['email'];
echo "Name: " . $name . "<br>";
echo "Email: " . $email;
```

PHP Form (using get method)

HTML:

```
<form action="process.php" method="get">
Name: <input type="text" name="name"><br>
Email: <input type="email" name="email"><br>
<button type="submit">Submit</button>
</form>
```

URL After Submitting:

http://localhost/process.php?name=Arefin&email=arefin@example.com

process.php

```
<?php
$name = $_GET['name'];
$email = $_GET['email'];
echo "Name: " . $name . "<br>";
echo "Email: " . $email;
```

Superglobals

- Superglobals are predefined built-in variables in PHP that are automatically available in every scope of a script.
- You can access them from anywhere in your program without using the global keyword.

$GLOBALS
$_SERVER
$_GET
$_POST
$_REQUEST
$_FILES
$_COOKIE
$_SESSION
$_ENV

$GLOBALS

$GLOBALS
Built-in variables that are always available in all scopes

Example:

```
<?php
$name = "Arefin";
function showName() {
	echo $GLOBALS['name'];
}
showName();  // Arefin
```

$_SERVER

$_SERVER
Provides information about the server and the current request.

Example:

```
<?php
echo $_SERVER['PHP_SELF'];  // /index.php
echo $_SERVER['REQUEST_METHOD'];  // GET or POST
echo $_SERVER['HTTP_HOST'];  // localhost or www.example.com
echo $_SERVER['REMOTE_ADDR'];  // 127.0.0.1 or 203.76.120.45
```

$_GET

$_GET
Used to receive data from the URL.

URL:
http://localhost/index.php?name=Arefin&age=25

Example:

```
<?php
echo $_GET['name'];
echo "<br>";
echo $_GET['age'];
```

Output:
Arefin
25

$_POST

$_POST
Used to receive data submitted from an HTML form using the POST method.

HTML

```
<form method="post">
<input type="text" name="username">
<button>Submit</button>
</form>
```

Example:

```
<?php
echo $_POST['username'];
```

$_REQUEST

$_REQUEST
Can retrieve data from GET, POST, and COOKIE (depending on PHP configuration).

Example:

```
<?php
echo $_REQUEST['username'];
```

Although it works, most developers prefer using $_GET or $_POST explicitly because it makes the source of the data clear.

$_FILES

$_FILES
Used for file uploads.

HTML:

```
<form method="post" enctype="multipart/form-data">
<input type="file" name="photo">
<button>Upload</button>
</form>
```

PHP:

```
<?php
print_r($_FILES);
```

$_COOKIE

$_COOKIE
Used to store small pieces of data in the user's browser.

Set Cookie:

```
<?php
setcookie("username", "Arefin", time() + 3600);
```

Read Cookie:

```
<?php
echo $_COOKIE['username'];
```

PHPSESSID (PHP System Cookie):
Created automatically when starting a session. It stores a unique random string on the browser, which points to session variables stored on the server.

$_SESSION

$_SESSION
Used to store user data on the server.

Start Session:

```
<?php
session_start();
$_SESSION['username'] = "Arefin";
```

Read Session Data:

```
<?php
session_start();
echo $_SESSION['username'];
```

$_ENV

$_ENV
Contains environment variables.

Example:

```
<?php
print_r($_ENV);
```

This is commonly used in frameworks like Laravel to access environment configuration

Include and Require

include()
include_once()
require()
require_once()

- All are used to include a file in another file. The main difference between them is how they handle errors and whether the file is included multiple times.

include() and include_once()

include:
If the file is not found, a warning is generated, but the script continues to execute. The file can be included multiple times.

include_once:
If the file has already been included, it will not be included again. If the file is not found, a warning is generated, but the script continues to execute.

require() and require_once()

require:
If the file is not found, a fatal error is generated and the script stops executing. The file can be included multiple times.

require_once:
If the file has already been included, it will not be included again. If the file is not found, a fatal error is generated and the script stops executing.

Important

- I am giving an Example to understand.
- For example, I am going to use include and require to my home page.

- I will use "include" in order to include some less important parts like testimonial, blog etc. into the home page. Because if any section has error, the complete page will not be into trouble to load. Only that particular section will have issue.

- I will use "require" in order to include the important parts like header, footer, navigation etc.

Cookie

What is a Cookie?

A cookie is a small piece of data that a website stores in the user's browser.

Why Do We Use Cookies?

- Remember logged-in users ("Remember Me")
- Store language preference
- Store dark/light theme
- Remember shopping cart items (small websites)
- Track user preferences

Cookie (Continue …)

Creating a Cookie:
setcookie(name, value, expire, path, domain, secure, httponly);

- Usually, you'll use only the first three parameters.

Example:
```
<?php
setcookie("username", "Arefin", time() + 3600);
echo "Cookie has been created.";
```

Cookie Name → username
Cookie Value → Arefin
Expires after → 3600 seconds (1 hour)

Cookie (Continue …)

Reading a Cookie:

```
<?php
echo $_COOKIE['username']; // Arefin
```

Check if a Cookie Exists:

```
<?php
if (isset($_COOKIE['username'])) {
	echo "Welcome " . $_COOKIE['username'];
} else {
	echo "Cookie not found.";
}
```

Cookie (Continue …)

Delete a Cookie:
To delete a cookie, set its expiration time to a time in the past.

```
<?php
setcookie("username", "", time() - 3600);
```

Important Rule:
setcookie() must be called before any HTML output.
So it is wrong:

```
<?php
echo "Hello";
setcookie("username", "Arefin", time() + 3600);
```

Cookie (Continue …)

Expiry Times:

time() + 60 // Expires after 1 min
time() + 3600 // Expires after 1 hour
time() + 86400 // Expires after 1 day
time() + (86400 * 30) // Expires after 30 days

Cookie (Continue …)

Practical Example:

Remember Me

login.php

```
<?php
if (isset($_COOKIE['username'])) {
        header("Location: dashboard.php");
        exit;
}
```

Cookie (Continue …)

```
if (isset($_POST['login'])) {
	$username = $_POST['username'];
	$password = $_POST['password'];
	// Dummy Login
	if ($username == "admin" && $password == "1234") {
		if (isset($_POST['remember'])) {
			// Remember for 7 days
			setcookie("username", $username, time() + (86400 * 7));
		}
		header("location: dashboard.php");
		exit;
	}
	echo "Invalid Username or Password";
}
```

Cookie (Continue …)

```
<form method="post">
    Username: <input type="text" name="username"> <br><br>
    Password: <input type="password" name="password"> <br><br>
    <input type="checkbox" name="remember"> Remember Me <br><br>
    <button type="submit" name="login">Login</button>
</form>
```

Cookie (Continue …)

dashboard.php

```
<?php
if (!isset($_COOKIE['username'])) {
        header("Location: login.php");
        exit;
}
echo "Welcome " . $_COOKIE['username'];
echo "<br><br>";
echo "<a href='logout.php'>Logout</a>";
```

Cookie (Continue …)

logout.php

```
<?php
setcookie("username", "", time() - 3600);
header("location: login.php");
exit;
```

This also works:

```
setcookie("username", "", time() - 1);
```

or

```
setcookie("username", "", 1);  // Means: January 1, 1970 00:00:01 UTC
```

Session

What is a Session?

- A session is a way to store user data on the server so that it can be accessed across multiple pages during the user's visit.
- Unlike cookies, session data is stored on the server, not in the user's browser.

Why Do We Use Sessions?

- User Login
- Admin Authentication
- Shopping Cart
- User Preferences
- Multi-step Forms
- Temporary User Data

Session (Continue …)

Important Concept:
What is saved in server and what is saved in browser.

On the Server:
Session ID : ABC123
username = Arefin
email = arefin@gmail.com
role = admin

In the Browser:
PHPSESSID = ABC123
The browser does not store: username, password, email. It only stores session id.

Session (Continue …)

Starting a Session:

```
<?php
session_start();
```

- session_start() must be called before any HTML output, just like setcookie().

Creating Session Variables:

```
<?php
session_start();
$_SESSION['username'] = "Arefin";
$_SESSION['email'] = "arefin@example.com";
echo "Session Created";
```

Session (Continue …)

Reading Session Variables:

```
<?php
session_start();
echo $_SESSION['username'];  // Arefin
echo "<br>";
echo $_SESSION['email'];  // arefin@example.com
```

Check if Session Exists:

```
<?php
session_start();
if(isset($_SESSION['username'])) {
	echo "Welcome " . $_SESSION['username'];
} else {
	echo "Please Login";
}
```

Session (Continue …)

Remove One Session Variable:

```
<?php
session_start();
unset($_SESSION['username']);
```

Destroy All Sessions:

```
<?php
session_start();
session_destroy();
```

Session (Continue …)

Practical Example:
Login and Logout

login.php

```
<?php
session_start();
if(isset($_POST['login'])) {
	$username = $_POST['username'];
	$password = $_POST['password'];
	if($username == "admin" && $password == "1234") {
		$_SESSION['username'] = $username;
		header("location: dashboard.php");
		exit;
	}
	echo "Invalid Username or Password";
}
```

Session (Continue …)

```
<form method="post">
    Username: <input type="text" name="username"> <br><br>
    Password: <input type="password" name="password"> <br><br>
    <button name="login">Login</button>
</form>
```

Session (Continue …)

dashboard.php

```
<?php
session_start();
if(!isset($_SESSION['username'])) {
        header("location: login.php");
        exit;
}
echo "Welcome " . $_SESSION['username'];
echo "<br><br>";
echo "<a href='logout.php'>Logout</a>";
```

Session (Continue …)

logout.php

```
<?php
session_start();
session_destroy();
header("location: login.php");
exit;
```

Cookies vs Sessions

Cookie	Session
Stored in the browser	Stored on the server
Limited storage (about 4 KB per cookie)	Can store much more data
Can be viewed or modified by the user	More secure because data stays on the server
Used for user preferences	Used for login and user authentication